

TRABAJO PRÁCTICO

INTEGRADOR

IA3.5 Redes de Datos

Tecnicatura Universitaria en Inteligencia Artificial

Cecilia Gisel Manno

Nancy Lucía Cima

Julia Sumiacher

Diciembre 2025

Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura

INTRODUCCIÓN

El presente trabajo implementa una arquitectura de tres capas (Cliente-Servidor1-Servidor2) para gestionar una biblioteca digital mediante una API RESTful. El proyecto integra los conceptos de comunicación de red, formato de datos JSON, y desarrollo de APIs estudiados durante la cursada.

Base de datos elegida: Archivo JSON con los 100 mejores libros de la historia según el Norwegian Book Club.

Sistema operativo: Windows 11

Entorno: Físico (desarrollo en máquina local)

Lenguaje: Python 3.11.9

Framework: FastAPI con uvicorn

ARQUITECTURA DEL SISTEMA

El sistema implementa una arquitectura de tres capas con separación de responsabilidades:

- **SERVIDOR 2 (Datos - Puerto 9000):** Gestiona el archivo JSON y las operaciones CRUD básicas.
- **SERVIDOR 1 (Gateway - Puerto 8000):** Actúa como proxy con autenticación Basic HTTP y rate limiting.
- **CLIENTE:** Interfaz CLI que se comunica únicamente con Servidor 1.

Ventajas de esta arquitectura:

- Separación de responsabilidades entre seguridad y datos
- Seguridad centralizada en el gateway
- Escalabilidad (Servidor 2 puede ser replicado)
- Los datos no están expuestos directamente al cliente

DESCRIPCIÓN DEL ENTORNO - SERVIDOR 2

Configuración

- **Sistema operativo:** Windows 11
- **Lenguaje:** Python 3.11.9
- **Framework:** FastAPI 0.109.0
- **Servidor ASGI:** uvicorn 0.27.0
- **Puerto:** 9000

Funcionalidad

Este servidor se encarga exclusivamente del almacenamiento y gestión de datos. No implementa seguridad ya que solo es accesible desde Servidor 1.

Endpoints implementados:

- GET / - Información del servidor
- GET /libros/ - Listar y filtrar libros
- GET /libros/{titulo} - Obtener libro específico
- POST /libros/ - Agregar libro
- PUT /libros/{titulo} - Actualizar libro
- DELETE /libros/{titulo} - Eliminar libro

Problemas encontrados

Inicialmente se consideró usar una sola capa servidor-cliente, pero la consigna requería implementar comunicación entre tres componentes. La solución fue crear una arquitectura de gateway que separa las responsabilidades de seguridad y datos.

Se encontro el problema de que para buscar diferenciaba mayusculas y minusculas, y no encontraba un libro si no estaba escrito exactamente igual, se corrigió en servidor 2 agregando case insensitive.

DESCRIPCIÓN DEL ENTORNO - SERVIDOR 1

Configuración

- **Sistema operativo:** Windows 11
- **Lenguaje:** Python 3.11.9
- **Framework:** FastAPI 0.109.0
- **Servidor ASGI:** uvicorn 0.27.0
- **Biblioteca HTTP:** requests 2.31.0
- **Puerto:** 8000

Funcionalidad

Este servidor actúa como gateway/proxy entre el cliente y el servidor de datos, implementando dos capas de seguridad:

1. Autenticación Basic HTTP:

- Usuario: admin
- Contraseña: redes2025
- Aplicable solo a POST y DELETE

2. Rate Limiting:

- Límite: 2 requests cada 15 segundos
- Basado en dirección IP del cliente
- Configurado con valores bajos para facilitar demostración

Problemas encontrados

Se probó inicialmente la biblioteca slowapi para rate limiting, pero presentaba incompatibilidades. Se optó por una implementación manual basada en diccionarios de timestamps que resultó más confiable y fácil de configurar.

DESCRIPCIÓN DEL ENTORNO - CLIENTE

Configuración

- **Sistema operativo:** Windows 11
- **Lenguaje:** Python 3.11.9
- **Biblioteca HTTP:** requests

Funcionalidad

El cliente proporciona una interfaz de línea de comandos (CLI) interactiva con las siguientes opciones:

- Buscar libro por título
- Filtrar libros por autor, idioma, país, rango de años
- Agregar libro (requiere autenticación)
- Actualizar libro
- Eliminar libro (requiere autenticación)

El cliente se comunica únicamente con Servidor 1 (puerto 8000) y no tiene conocimiento de la existencia de Servidor 2.

Problemas encontrados

Inicialmente el manejo de errores en input del usuario causaba excepciones. Se implementó validación con try-except para entrada vacía o no numérica en el menú.

DESCRIPCIÓN DE LA BASE DE DATOS

El sistema utiliza un archivo JSON que contiene información sobre los 100 mejores libros de la historia según el Norwegian Book Club.

Estructura de datos

- **Cantidad de objetos:** 100 libros (más los agregados durante pruebas)
- **Formato:** Array de objetos JSON

Propiedades de cada objeto

- **author** (string): Nombre del autor
- **country** (string): País de origen
- **imageLink** (string): Ruta de imagen de portada
- **language** (string): Idioma original
- **link** (string): Enlace a Wikipedia
- **pages** (integer): Número de páginas
- **title** (string): Título del libro
- **year** (integer): Año de publicación (puede ser negativo)

Ejemplo de entrada

```
{
  "author": "Chinua Achebe",
  "country": "Nigeria",
  "imageLink": "images/things-fall-apart.jpg",
  "language": "English",
  "link": "https://en.wikipedia.org/wiki/...",
  "pages": 209,
  "title": "Things Fall Apart",
  "year": 1958
}
```

PROCEDIMIENTO Y EJEMPLOS DE USO

Ejecución del sistema

Terminal 1 - Servidor 2:

```
cd servidor2
python -m uvicorn main:app --port 9000
```

Terminal 2 - Servidor 1:

```
cd servidor1
python -m uvicorn main:app --port 8000
```

Terminal 3 - Cliente:

```
cd cliente
python main.py
```

Ejemplos de peticiones API

GET - Obtener información del servidor:

```
curl http://localhost:8000/
```

Respuesta:

```
{"mensaje": "Bienvenido a la Biblioteca Online",
 "servidor": "Servidor 1 - Gateway",
 "cantidad_libros": 103,
 "servidor_datos": "activo"}
```

GET - Buscar libro específico:

```
curl http://localhost:8000/libros/Things%20Fall%20Apart
```

GET - Filtrar libros:

```
curl 'http://localhost:8000/libros/?autor=Homer&idioma=Greek'
```

POST - Agregar libro (CON autenticación):

```
curl -X POST -u admin:redes2025 \
'http://localhost:8000/libros/Mi%20Libro?autor=David&
idioma=Spanish&paginas=200&pais=Argentina&anio=2025'
```

DELETE - Eliminar libro (CON autenticación):

```
curl -X DELETE -u admin:redes2025 \
http://localhost:8000/libros/Mi%20Libro
```


Verificación de seguridad

1. POST sin autenticación (debe fallar con 401):

```
curl -X POST 'http://localhost:8000/libros/Test?...
```

Respuesta esperada:

```
{"detail": "Credenciales incorrectas"}
```

2. Rate limiting (5 requests rápidos):

Desde el navegador en <http://localhost:8000/docs>:

- Request 1: 200 OK
- Request 2: 200 OK
- Request 3: 429 Too Many Requests
- Request 4: 429 Too Many Requests
- Request 5: 429 Too Many Requests

Respuesta del rate limit:

```
{"detail": "Máximo de intentos 2 per 15 second excedido"}
```

INSTRUCCIONES DE USO DEL CLIENTE

Requisitos previos

- Python 3.11 o superior instalado
- Biblioteca requests instalada (pip install requests)
- Servidor 1 y Servidor 2 ejecutándose

Ejecución

```
cd cliente  
python main.py
```

Uso del menú interactivo

Al ejecutar el cliente, aparecerá un menú con 6 opciones:

Opción 1 - Buscar libro:

Permite buscar un libro por su título exacto. Ingrese el título completo.

Opción 2 - Filtrar libros:

Permite filtrar por autor, idioma, país y/o rango de años. Todos los filtros son opcionales (presione Enter para omitir).

Opción 3 - Agregar libro:

Requiere autenticación. Complete todos los campos requeridos (título, autor, idioma, páginas, país, año). Los campos imagen y link son opcionales.

Opción 4 - Actualizar libro:

Ingrese el título del libro a actualizar y los nuevos datos.

Opción 5 - Eliminar libro:

Requiere autenticación. Ingrese el título exacto del libro a eliminar.

Opción 6 - Salir:

Finaliza la ejecución del cliente.

Credenciales de autenticación

Las credenciales están configuradas en el código del cliente:

- **Usuario:** admin
- **Contraseña:** redes2025

CONCLUSIONES

El proyecto logró cumplir exitosamente con todas las etapas propuestas:

- Implementación de arquitectura de tres capas con separación de responsabilidades
- Comunicación efectiva entre Cliente-Servidor1-Servidor2
- Autenticación Basic HTTP en endpoints críticos (POST/DELETE)
- Rate limiting funcional para prevención de abuso
- Gestión completa de operaciones CRUD sobre base de datos JSON

La arquitectura de tres capas demostró ventajas significativas en escalabilidad y mantenibilidad comparada con una arquitectura simple cliente-servidor. La separación entre gateway de seguridad y servidor de datos permite modificar o reemplazar componentes de forma independiente.

El uso de FastAPI facilitó la implementación de una API RESTful robusta con documentación automática (Swagger UI), lo que agilizó las pruebas y validación del sistema.

REFERENCIAS

1. FastAPI Documentation. <https://fastapi.tiangolo.com/> (Consultado: Diciembre 2025)
2. Python Requests Library. <https://requests.readthedocs.io/> (Consultado: Diciembre 2025)
3. HTTP Basic Authentication - RFC 7617. <https://tools.ietf.org/html/rfc7617>
4. Rate Limiting Patterns in API Design.
<https://cloud.google.com/architecture/rate-limiting-strategies-techniques>
5. Material didáctico de la cátedra IA3.5 Redes de Datos - FCEIA UNR